Lantern Edge AI Plan

A small on-device AI that uses Lantern as its source of truth · USB, laptop and phone · 19 October 2026

> **Goal.** Anyone, anywhere, with no internet, no account and no cost, gets an assistant that is **accurate** (answers come from Lantern's checked tools and content, not the model's memory), **safe and moral** (a standard aligned model inside Lantern's rules), and **useful** (it does the calculation, writes the letter, finds the lesson). Delivered three ways from one GitHub project: a website that installs on a phone, a laptop launcher, and a 32 GB USB stick that copies to many.

32 GB USB / laptop	Android phone	iPhone	One download button
High — proven pattern; Lantern already speaks the server's API.	**High** — web app now; APK optional (developer verification from 2027).	**Medium** — web app only (no app installs from GitHub); iOS 26+ for GPU.	**Partial** — one button per device; "download once, copy many".

1. The key decision: a small model, a growing toolset

A small model cannot be trusted to *know* things. On Artificial Analysis's knowledge test, Qwen3.5-4B answered 12.8% correctly and, when it did not know, invented an answer 80% of the time; the 9B reached only 14.7%. On edge hardware, size does not buy truth. Grounding does.

In Lantern the model's job is narrow: understand the question, pick the right tool card, fill in the values, explain the verified result in plain words, and decline or redirect when it should. Published evaluations put reliable single-step tool use at about 1–3B parameters; below about 1B, multi-step and nested tool calls fail. So:

- **Keep the model as small as the Bench allows** (section 6), separately for each device tier.
- **Grow the toolset**: more tool cards, more checked formulas, and more reference content the model searches instead of remembering.

Model tiers (4-bit GGUF; one file format for every engine)

Device	RAM	Model	File	Role
Basic phone	4 GB	Qwen3.5-0.8B Q4_K_M	533 MB	Rephrase and pick from cards; cards do the work
Typical phone	6–8 GB	Qwen3.5-2B Q4_K_M	1.28 GB	Full tool use, plain-language answers
8 GB laptop	8 GB	Qwen3.5-4B (4-bit)	≈ 3 GB	Multi-step tool use, writing help
16 GB laptop	16 GB+	Qwen3.5-9B (4-bit)	≈ 6 GB	Hardest questions; classroom server
Alternative	8 GB+	Gemma 4 E2B Q4_0	2.84 GB	Native function calling; Bench decides

Qwen3.5 small models are Apache 2.0 (Artificial Analysis Intelligence Index: 0.8B = 9, 2B = 16, 4B = 27, 9B = 32). Gemma 4 GGUF builds are published as Apache 2.0. Use standard instruction models, never "abliterated" ones, and keep "thinking" mode off on phones (every extra token costs time). In-browser engines load at most 2 GB per file, so phones use the 0.8B and 2B files; larger models are split into pieces, which llama.cpp loads natively.

Toolset and content budget

Layer	Today	Phone budget	USB budget
Lantern code, tools, lessons	1.36 MB	< 20 MB	< 20 MB
AI engine (browser)	—	≈ 9 MB (2.4 MB compressed)	same, plus llama.cpp for Win/Mac/Linux < 0.5 GB
Reference packs (searched, cited)	—	0.2–1 GB curated	≈ 12 GB+ (e.g. intro-only English Wikipedia was 12 GB in 2022)
Models	—	0.5–1.3 GB	≈ 11 GB (all four tiers)
Total		**≈ 1–2.5 GB**	**≈ 25 GB of 32 GB** (format exFAT)

2. Architecture: one Lantern, three engines

Person
asks in plain words, in English or Filipino (Lantern's other languages later)

↓

Lantern Assist / Chat
Card matcher runs first (no AI needed for a known question). Builds the tool prompt (≈ 950 tokens), checks the model's reply, runs the tools, shows the source.

↓ one engine interface: `Lantern.ai`

On this device
llama.cpp compiled to WebAssembly (wllama) with WebGPU. Runs inside the page. Phones, any modern browser.

Local server
`llama-server` on a laptop or USB stick serves Lantern and the model on the same address; also shares to phones on a local hotspot.

Courier
Copy the prompt into any AI app and paste the reply back. Works where nothing else can.

↓ the model only proposes; Lantern decides

Lantern tools = the truth
FORMULA engine and 16+ domain cell functions, prescripted tool cards, lessons, Compose checker, reference packs. Forge self-tests guard every tool.

How accuracy, safety and usefulness are enforced (not hoped for)

1. **Cards first.** Known question types are matched without the model; the model fills values or rephrases.
2. **Constrained output.** llama.cpp can force the reply into a fixed format, so even a 0.8B model cannot emit a malformed tool call.
3. **Tool results are final.** The answer must quote computed values; Lantern recomputes and rejects mismatches.
4. **Retrieval with citations.** Facts come only from Lantern content, with the source shown; otherwise "not in my library".
5. **Sensitive topics route to sourced lessons** (medicine doses, electrical work, chemicals), never to free generation.
6. **Aligned models only**, plus Lantern's own rules in the system prompt; no refusal-stripped models.

7. **Nothing auto-applies.** Forge proposals are staged and gated by self-tests, as today.

3. Deployment targets

Target	How it works	Constraints
Website / phone web app (all phones)	Lantern + engine on GitHub Pages (or any static host). Open once online, Add to Home Screen, download the model once on Wi-Fi (or load it from a file copied from a USB stick or laptop). Afterwards fully offline.	Needs HTTPS hosting. GPU path needs iOS 26+ / recent Android Chrome; otherwise slower CPU path. Browser storage quota for a 0.5–1.3 GB model must be tested per phone.
Laptop / USB	Launcher per OS starts `llama-server --path lantern`, picks the model by RAM, opens the browser. Lantern detects the server on its own address.	x86-64 Windows/Linux or Mac. Graphics-card fallbacks needed (facts' changelog documents the crashes).
Classroom	One laptop runs the server with `--host 0.0.0.0` on a hotspot; phones open Lantern from it and share its model.	Plain HTTP on a LAN disables some browser features; the server path does not need them.
Android app (later)	APK on GitHub Releases: Lantern in a WebView plus native llama.cpp (faster than the browser).	Sideloading works today; Google's developer verification goes global in 2027 (register as a verified developer).
iPhone app (optional)	App Store/TestFlight build; could use Apple's built-in on-device model on iPhone 15 Pro and later.	No installs from GitHub outside the EU; Apple developer account required.

4. Distribution: GitHub, then copy

- **GitHub Releases** hold the packs: each file under 2 GiB, up to 1,000 files per release, no limit on total size or bandwidth. Larger models are split into pieces. GitHub's "Download ZIP" does not include release files, so it is not the button.
- **Landing page** (GitHub Pages) with one button per device: *Use on this phone*, *Get the laptop kit*, *Make a Lantern USB*.
- **Download once, copy many.** A 25 GB (or even 2 GB) download over Philippine mobile data is the wrong primary path. One person with good internet builds a USB stick; sticks are copied; phones take the model from a stick (USB-C adapter) or from a laptop over local Wi-Fi. The phone app accepts a model file from storage for exactly this reason.
- **Licences.** Lantern is CC0; llama.cpp and wllama are MIT; Qwen3.5 is Apache 2.0. Ship each licence beside its file.
- **Hosting note.** The repository is public, so GitHub Pages is free. If it is made private, Pages needs a paid plan.

5. Phases

Phase	Deliverable	Done when
1. Test build (now)	Lantern with the `Lantern.ai` engine layer: on-device engine (wllama), local-server auto-detect, Courier. Hostable site folder (offline service worker, install manifest). Laptop launchers. Lantern Bench v0 runnable in the app.	Adam runs it on real phones and a laptop and reports speed, memory and Bench scores per model.
2. Tune	Constrained output; more tool cards; first curated reference packs with search and citations; Filipino questions in the Bench.	The chosen phone model passes the Bench thresholds.
3. USB kit v1	All model tiers, engines for three OSes, reference packs, phone packs to copy, one “make a USB” script with checksums.	Clean stick works on Windows, Mac and Linux machines in the field.
4. Apps (optional)	Android APK (native llama.cpp); iPhone App Store build if worthwhile.	Faster than the web app on the same phone.
5. Growth	Forge-driven tool growth, every change gated by self-tests and the Bench.	Bench score rises without a bigger model.

6. Lantern Bench: how the size question gets answered

A fixed set of real questions with tool-verified answers, run against each candidate model on each device tier. The smallest model that passes is the one that ships for that tier, and the Bench is re-run whenever the toolset grows.

- **Tool accuracy:** did the model pick the right card and values, and does Lantern’s computed answer match the expected value?
- **Format reliability:** share of replies Lantern could parse without a retry.
- **Grounding:** for questions Lantern cannot answer, does it say so instead of inventing?
- **Safety routing:** do sensitive questions go to sourced lessons?
- **Speed and memory:** time to first answer, tokens per second, whether the phone kept the app alive.

Proposed thresholds to start: at least 90% tool accuracy on single-step questions, 95% parseable replies, 100% safety routing, and a first answer within 20 seconds on the target phone. They are starting points to revise once real numbers exist.

7. Risks and open questions

- **Phone browser memory.** How large a model a browser tab can hold before the phone closes it. Phase 1 measures it.
- **Safari GPU support** for the in-browser engine may need a compatibility mode; the CPU path is the fallback.
- **Qwen3.5-0.8B** is reported to loop more than larger models; sampling settings and output limits contain it.
- **Model hosting.** The test build downloads models from Hugging Face; field kits carry them on the stick. Mirror on GitHub Releases for resilience.
- **Android 2027 verification** affects the optional APK, not the web app.

- **Mobile data cost** argues for copying over downloading; the design supports both.

8. What Adam tests in Phase 1

1. Open the test site on each phone (note model, RAM, OS version, browser). Add to Home Screen.
2. Engine check: Lantern reports whether the GPU path is available.
3. Download Qwen3.5-0.8B on Wi-Fi; run three questions; note load time, answer time and whether the app stayed open. Repeat with Qwen3.5-2B.
4. Turn on airplane mode, close and reopen: Lantern and the model should still work.
5. Run Lantern Bench; export the results file and send it back.
6. On a laptop, run the launcher and repeat steps 3 and 5 with the 4B model.

Sources: Artificial Analysis, “Qwen3.5 small models”; Google AI, Gemma 4 model overview; Hugging Face model pages unsloth/Qwen3.5-0.8B-GGUF, unsloth/Qwen3.5-2B-GGUF, ggml-org/gemma-4-E2B-it-GGUF; Digital Applied, “Small language models for on-device agents (2026)”; GitHub Docs, “About releases”; Android Authority, developer verification rollout (1 Oct 2026); App Developer Magazine, “WebGPU in iOS 26”; Kiwix, “What is the size of Wikipedia?” (2022); wllama 3.8.1 package; llama.cpp source (server CORS and static path), 19 Oct 2026; WEAREOSE/facts launchers and changelog.